

# DFIR Chat System: A Sovereign, On-Premise AI System for Digital Forensics

---

An agentic, offline-first platform that lets investigators interrogate mobile-device evidence in natural language — and auto-generate crime-driven case summaries — without a single byte leaving the workstation.

Many specialised scripts. One reasoning brain. Deterministic by default; generative only when it must be.

---

**Prepared for**

Government & law-  
enforcement  
digital forensics  
stakeholders

**Issued**

June 2026

**Author**

FCTM - DFIR Chat  
System Team

**Handling**

Restricted distribution

# 1 Executive Summary

---

Trinity is a self-contained artificial-intelligence system built for one purpose: to help digital-forensics examiners make sense of the growing mountain of mobile-device evidence — chat logs, calls, contacts, media metadata — quickly, accurately, and *without exposing that evidence to any external service*. It runs entirely on a single Windows workstation, with every model, index, and computation held locally. It is designed to operate fully offline, including on air-gapped machines inside a secure facility.

Two capabilities sit on top of the same engine. The first is a **conversational question-and-answer interface**: an examiner asks questions in plain English — "how many distinct chats are in this dump?", "were any passwords shared?", "what was the longest call and with whom?" — and receives precise, source-cited answers. The second is an **automated forensic summary** that reads an entire extraction and produces a crime-driven narrative of what the device reveals, organised around the offence categories an investigator cares about.

What distinguishes Trinity from a generic "chatbot over documents" is its architecture. Rather than handing every question to a large language model and hoping for the best, Trinity routes each query through a **cascade of deterministic specialists**. Counting, ranking, date-range, and relationship questions are answered by exact database queries that read values directly from the evidence and therefore *cannot hallucinate*. Only genuinely open-ended, analytical questions reach the language model — and even then, an automatic verification step withholds any answer it cannot ground in the retrieved evidence. The result is a system that is auditable and reproducible where it counts, and flexible where flexibility adds value.

## AT A GLANCE

- **Deployment:** single Windows workstation; runs fully offline / air-gap capable; no cloud dependency, no data egress.
- **Reasoning model:** Qwen3-14B-Instruct (Q4\_K\_M, llama.cpp) running locally on the GPU.
- **Retrieval model:** BAAI BGE-M3 multilingual embeddings (1,024-dimension), generated on-device.
- **Compute:** NVIDIA RTX 4060 (8 GB) with CUDA 13 / cuDNN 9; full GPU offload; scales to higher-VRAM cards.
- **Architecture:** seven-tier query router — six deterministic tiers (exact SQL) ahead of one guarded generative tier.
- **Validated accuracy:** ~96% (Android) / ~95% (iOS) on forensic-summary fact checks; the deterministic tiers answered 81 of 110 questions on the iOS Q&A suite with 50/52 of the checkable answers verified correct.

## 2 The Problem Trinity Addresses

---

Mobile devices have become the single richest source of evidence in modern investigations. A single phone extraction routinely contains tens of thousands of messages, calls, and contacts across multiple applications. Examiners are expected to find the relevant needle — a shared credential, a location pin, a money reference, a contact relationship — under time pressure and to a standard that will survive scrutiny in court.

Two forces collide here. On one side, AI language models are obviously useful for sifting natural-language evidence at scale. On the other, the dominant way to access those models — sending data to a cloud API — is a non-starter for sensitive casework. Evidence is subject to chain-of-custody obligations, data-sovereignty rules, and, in many government contexts, classification controls. Uploading a suspect's chat history to a third-party server is not an option, and even self-hosted cloud carries unacceptable exfiltration risk.

A second, subtler problem is trust in the answer. General-purpose chatbots are prone to *hallucination* — producing fluent, plausible, and wrong statements. In a forensic context a confidently fabricated figure or a misattributed quote is worse than no answer at all. Investigators need outputs they can defend: reproducible, traceable to specific source rows, and conservative when the evidence is thin.

Trinity is built around both constraints simultaneously. It keeps everything local so the privacy problem never arises, and it puts deterministic, value-reading database logic *ahead of* the language model so that the trust problem is contained to the narrow set of questions that genuinely require open-ended reasoning.

## 3 System Overview: Two Tools, One Engine

---

From the examiner's seat, Trinity presents two workflows. Both are driven from a desktop analyzer application and share the same underlying knowledge base and reasoning engine.

### 3.1 Conversational Q&A

The examiner loads a processed extraction and asks questions in natural language. Trinity holds a short conversation memory (the last three exchanges), so follow-ups such as "and how many of those were outgoing?" resolve against the previous turn. Every answer carries the specific source rows it was derived from, so a finding can be opened and inspected. A single session can hold several cases at once: Trinity automatically loads every built knowledge base it finds, and the examiner switches the active case in-session with a simple `use <case>` command.

### 3.2 Automated Forensic Summary

With one action — the analyzer's **Summarize** button — Trinity reads the whole extraction and produces a structured forensic narrative. The summary is **crime-driven**: it leads with a breakdown of detected offence indicators organised into the categories an investigator works in, then provides a broader activity overview (the people, chats, accounts, and identifiers present), and closes with a measured conclusion. Where no offence indicators are detected, it says so plainly rather than inventing them.

### 3.3 Part of a wider forensic toolkit

Trinity is orchestrated by a C# analyzer user interface that also drives sibling on-premise tools — including face-clustering and image-classification engines — giving examiners a single console for text, call, and media evidence. This white paper focuses on Trinity's text/communications engine.

## 4 The Agentic Architecture: Many Scripts, One Brain

---

Trinity is composed of a dozen specialised Python modules — an ingest pipeline, a canonical-schema normaliser, several deterministic query tiers, a hybrid search engine, and a summary generator. They are not a loose collection of utilities; they are wired into a single decision flow that behaves like one reasoning brain. The organising principle is

simple and strict: **for any question, the most specific tier that can answer it exactly wins; if a tier cannot answer, it steps aside and passes the question down.** The language model is the last resort, not the first.

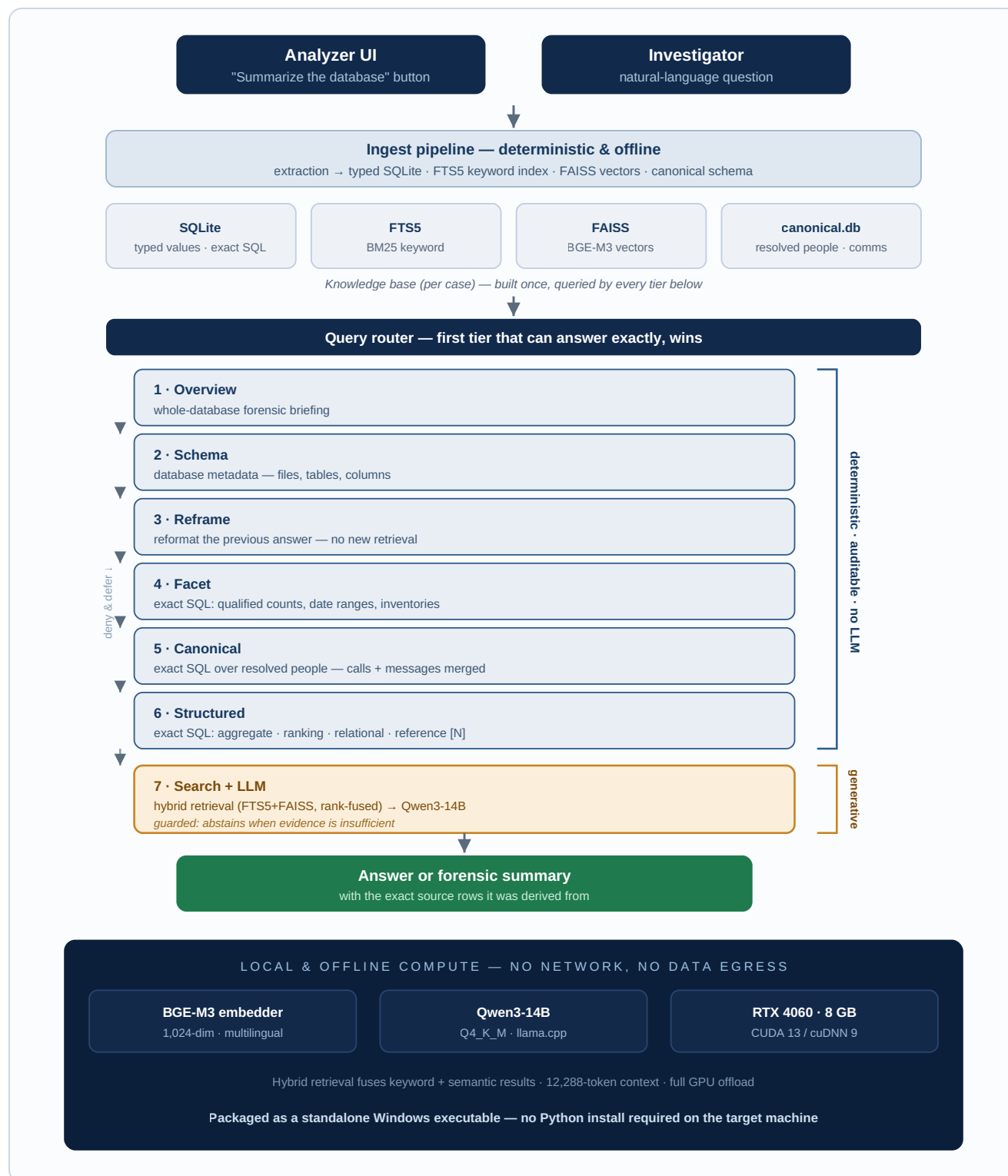


Figure 1. Trinity's unified query flow. Evidence is ingested once into a four-store knowledge base; every question then descends a cascade of deterministic tiers, reaching the language model only when no exact tier applies.

## 4.1 How a question travels through the brain

When a question arrives, Trinity evaluates it against each tier in a fixed precedence order. The first three tiers handle intent that is obvious from the wording. An explicit request to summarise the whole database routes to the **Overview**

tier; a question about the dataset's own structure (file counts, tables, columns) routes to the **Schema** tier; and a request to reshape the last answer ("say that more briefly", "expand on that") routes to the **Reframe** tier, which reworks the previous response without touching the evidence again.

If none of those apply, the question enters the three deterministic data tiers. The **Facet** tier is the most specific: it answers qualified, aggregate-shaped questions — "how many missed calls", "incoming versus outgoing", "what is the date range", "what documents are present" — with exact SQL, so these never get mis-handled by a broader template or dropped into fuzzy search. The **Canonical** tier answers questions about people by reading a normalised schema in which identities have been resolved and calls and messages are merged per party. The **Structured** tier covers general aggregation, ranking, relational ("did X contact Y"), and back-reference questions ("tell me more about entry [3]") by resolving against the previous result set. Each of these tiers reads values directly from the database; none of them can invent a number.

Only when every deterministic tier has declined does the question reach the **Search + LLM** tier. Here a hybrid retriever combines keyword (BM25) and semantic (vector) search, fuses the two ranked lists, and passes the most relevant evidence rows to the language model, which composes a prose answer grounded in that evidence. This is the one tier that generates language — and it is wrapped in the safeguards described in §6.

| Tier             | Method                          | What it answers                                               |
|------------------|---------------------------------|---------------------------------------------------------------|
| 1 • Overview     | Deterministic + LLM narration   | Whole-database forensic briefing on request.                  |
| 2 • Schema       | Deterministic                   | Dataset structure — file counts, tables, columns.             |
| 3 • Reframe      | LLM (no re-retrieval)           | Reformatting the previous answer only.                        |
| 4 • Facet        | Exact SQL                       | Qualified counts, type breakdowns, date ranges, inventories.  |
| 5 • Canonical    | Exact SQL                       | Questions about resolved people; calls + messages merged.     |
| 6 • Structured   | Exact SQL                       | Aggregation, ranking, relational, and back-reference queries. |
| 7 • Search + LLM | Hybrid retrieval<br>→ Qwen3-14B | Open-ended analytical questions, with abstention guard.       |

#### Why this design matters for evidence

Because six of the seven tiers compute their answers with deterministic SQL over the original values, the bulk of an examiner's questions are answered without any generative model in the loop. Those answers are reproducible to the byte and traceable to specific rows — the qualities an answer needs to withstand challenge.

## 5 Models & Compute Stack

Every model Trinity uses is open-weight and runs locally. Nothing is downloaded at run time; the system explicitly forces offline mode so it behaves identically on an air-gapped machine.

### 5.1 The reasoning model

Trinity's language model is **Qwen3-14B-Instruct**, run through llama.cpp in the efficient **Q4\_K\_M** GGUF quantisation. This quantisation was selected after A/B testing against a smaller one: it matched on accuracy while producing cleaner, safer output. The model runs with a 12,288-token context window and full GPU offload. Crucially, Trinity's accuracy does not hinge on the size of this model — because the deterministic tiers do the exact-fact work, the language model is used only for open-ended phrasing, where a 14B local model is more than sufficient.

### 5.2 The retrieval model

Semantic search is powered by **BAAI BGE-M3**, a strong multilingual embedding model that produces 1,024-dimension vectors. Multilingual capability matters in real casework, where chats mix languages and scripts. Embeddings are computed on-device and indexed with FAISS; keyword search uses SQLite's FTS5 (BM25). The two are combined with **Reciprocal Rank Fusion**, with a modest boost toward exact keyword hits so that the precisely-named row is not buried beneath merely-similar ones.

### 5.3 Hardware and packaging

The reference deployment runs on an NVIDIA RTX 4060 (8 GB) with the CUDA 13 / cuDNN 9 stack. Trinity is delivered as a **standalone Windows executable** built with PyInstaller; the CUDA and cuDNN runtime libraries are bundled alongside it, so a target machine needs no Python environment and no separate driver-level installation beyond the GPU driver itself. The same code scales to higher-VRAM GPUs, which allows the 14B model to sit entirely in video memory or larger models to be hosted as hardware allows.

| Component        | Specification                                                            |
|------------------|--------------------------------------------------------------------------|
| Reasoning model  | Qwen3-14B-Instruct · Q4_K_M GGUF · llama.cpp runtime                     |
| Context window   | 12,288 tokens · full GPU offload (all layers)                            |
| Embedding model  | BAAI BGE-M3 · 1,024-dimension · multilingual · on-device                 |
| Vector index     | FAISS (dense) fused with SQLite FTS5 / BM25 (keyword)                    |
| Fusion           | Reciprocal Rank Fusion with keyword boost                                |
| Structured store | SQLite with detected column types + a fixed canonical schema             |
| GPU / runtime    | NVIDIA RTX 4060 8 GB · CUDA 13 · cuDNN 9 (scales to higher-VRAM cards)   |
| Packaging        | Standalone Windows .exe (PyInstaller); CUDA/cuDNN bundled                |
| Network          | Fully offline by default (Hugging Face / transformers offline flags set) |

## 6 Privacy, Trust & Data Sovereignty

For a government deployment these properties are not features; they are preconditions. Trinity was designed so each one is structural rather than a matter of configuration discipline.

## 6.1 Nothing leaves the workstation

All models, indexes, and computation are local. Trinity sets its dependencies to offline mode at start-up, so it makes no outbound network calls even if a network is present, and it runs unchanged on an air-gapped machine. Evidence stays under the same custody controls as the workstation it lives on, which keeps chain-of-custody and data-sovereignty obligations intact and removes the third-party-exfiltration risk inherent in cloud AI.

## 6.2 Determinism and auditability

The six deterministic tiers compute answers from the original values with plain SQL, so the same question over the same evidence returns the same answer every time — independent even of which quantisation of the model is installed (the deterministic result is "method, not model"). Trinity also keeps append-only operational logs of each query — intent, retrieval counts, timing, and the rows returned — providing a reviewable trail of how a given answer was produced.

## 6.3 Guarding the generative tier

The single generative tier is fenced in two ways. First, it only ever sees evidence rows retrieved from the case, never free-floating model knowledge. Second, a verification step checks the proper-noun content of the drafted answer against the retrieved evidence; if the majority of the distinctive phrases in the answer cannot be supported by the source rows, Trinity withholds the answer and abstains rather than risk a fabricated finding. The system is deliberately built to prefer "not enough evidence" over a confident guess.

### Design posture

Deterministic by default; generative only when necessary; abstaining when unsure. In a forensic setting, a withheld answer is recoverable — a fabricated one may not be.

# 7 Accuracy & Validation

---

Trinity has been validated against real mobile-extraction knowledge bases for both Android and iOS WhatsApp evidence, using fixed question suites scored against ground truth read directly from the underlying database.

## 7.1 Conversational Q&A

On a 110-question iOS test suite, the deterministic tiers alone answered **81 of 110** questions outright. On the subset whose answers could be mechanically checked against the database, **50 of 52** were verified correct — and the two apparent misses were artefacts of the test harness rather than wrong answers, making the effective result 52 of 52. The remaining 29 questions were genuinely analytical and were the only ones routed to the language model. This is the architecture working as intended: the great majority of questions resolved by exact, reproducible logic, with the model reserved for the open-ended remainder.

That validation run also drove concrete fixes that are now in production — for example, preventing a forensics reference document inside one dump from drowning out the actual chat evidence, and correcting count questions that had previously collapsed to a single grand total. Capability examples that now resolve correctly include distinct-chat counts, media-type breakdowns, earliest/latest message dates, longest and shortest calls with the counterpart named, shared-credential and shared-location detection, and business-account labels.

## 7.2 Forensic summary

The automated summary was validated at approximately **96% factual accuracy on Android** and **~95% on iOS**, measured against the source database, with complete table coverage (every populated table surfaced in both the narrative and its citation list). The summary's deterministic section builders extract the facts; the language model only narrates them, which is why the factual content holds up under checking.

| Validation                                 | Result                                                           |
|--------------------------------------------|------------------------------------------------------------------|
| iOS Q&A suite — answered deterministically | 81 of 110 questions (no LLM)                                     |
| iOS Q&A suite — checkable answers verified | 50 / 52 correct (effectively 52 / 52)                            |
| iOS Q&A suite — routed to the LLM          | 29 (genuinely analytical only)                                   |
| Forensic summary — Android                 | ~96% factual accuracy vs. database                               |
| Forensic summary — iOS                     | ~95% factual accuracy vs. database                               |
| Demonstrated corpora                       | iOS WhatsApp ≈ 39,300 indexed segments; Android WhatsApp ≈ 2,470 |

Figures reflect internal validation runs on the stated test suites and corpora; they are indicative of system behaviour on comparable WhatsApp extractions rather than a certified benchmark across all device types.

## 8 Deployment & Outlook

Trinity deploys as a single executable onto a Windows workstation with a supported NVIDIA GPU. Cases are processed once into knowledge bases and then queried interactively or summarised on demand; multiple cases can be held in one session. Because the system is schema-aware rather than hard-coded to one data layout, the same build serves multiple iOS and Android cases without modification, identifying the structure of each extraction by its schema rather than its file names.

The near-term roadmap follows two threads. The first is **breadth of evidence** — extending the format-driven ingest to additional applications and device types beyond WhatsApp, and deepening the crime-indicator taxonomy that drives the summary. The second is **capacity** — Trinity already pools video memory across GPUs, and a planned move to higher-VRAM hardware will let the current model run wholly in video memory and allow substantially larger local models to be hosted where a case justifies it. Both threads preserve the core commitment: everything stays on-premise.

## 9 Conclusion

Trinity demonstrates that the benefits of modern AI can be brought to sensitive digital-forensics work *without* the privacy and trust compromises that usually come with them. By keeping every model and computation on the examiner's own machine, it removes the data-exposure problem entirely. By placing a cascade of deterministic, value-reading database tiers ahead of the language model — and guarding that model with evidence-grounding and



abstention — it keeps the trust problem contained to the narrow band of questions that truly need open-ended reasoning. Many specialised scripts, coordinated into one disciplined reasoning brain, give investi